

一、python 规范

1. 命名规则

- (1) 必须以下划线或字母开头
- (2) 前后单下划线为系统变量
- (3) 双下划线开头为类的私有变量
- (4) 关键字不能用作变量名

2. 注释

- 单行注释以 # 开头
- 多行注释可用多个 # 或者用 三引号 (文档注释)

3. 多行语句

- 行尾使用反斜线 (\) 来续行

4. 同一行写多条语句

- 语句间用分号 (;) 分隔

二、输入输出

1. 输出 print ()

- print 默认在末尾换行

a = 1

b = 2

c = 3 # 直接输出多个变量 print(a , b , c)

输出: 1 2 3

使用 end 参数用指定的字符替换末尾的换行符 print(a , end = ' @')

输出: 1@

使用 formatprint(' a={}' ' . format(a)) # 输出:

a=1print(' a={0} , b={1} , c {2}' ' . format(a , b , c)) # 输出: a=1,



b=2, c3

2. 输入 input ()

- input 输入的始终是字符串， 需要转换为其他数据类型使用

三、python 数据类型

1. 六个标准数据类型

- (1) Number (数字)
- (2) String (字符串)
- (3) List (列表)
- (4) Tuple (元组)
- (5) Sets (集合)
- (6) Dictionary (字典)

2. Number

- 包括: int (整型) 、 float (浮点型) 、 bool (布尔型) 、 complex (复数) 、 long (长整型)
- 清楚哪些值转换为布尔类型后值是 False

```
print(bool([]))
```

```
# 输出: Falseprint(bool(' '))
```

```
# 输出: Falseprint(bool({}))
```

```
# 输出: Falseprint(bool(()))
```

```
# 输出: False# 注意下面两个的区别
```

```
print(bool(0))
```

```
# 输出: Falseprint(bool('0'))
```

```
# 输出: True
```

- 浮点数的内置函数运算会有误差， 也就是小数的精度问题

3. String



字符串属于序列，除此之外还有：元组、列表（集合和字典不是序列类型）

单引号和双引号可以互换，也可以互嵌

三引号表示多行字符串（也可以作为文档注释）

另外：三引号内可以直接使用回车、制表符，可以不使用转移字符来表示

字符串常用操作：

（1）连接和重复

```
print('hello' * 3, ' world' + 'ld')
```

输出： hellohellohello world

字符串的切片（左闭右开）

```
word = 'hello world' print(word[0: 5])
```

输出： helloprint(word[: 5])

输出： helloprint(word[1:])

输出： ello worldprint(word[:])

输出： helloworldprint(word[0: 5: 2])

输出： hloprint(word[2: -2])

输出： lloworprint(word[-2: 2])

输出空串

（2）转义字符

- 要注意这种复杂的转义字符一起输出

在字符串内的“\r”、“\t”、“\n”等字符，会转换为空白字符（回车符、水平制表符、换行符……）

```
printf('hello\tworld') # 输出： hello world
```

（3）Raw 字符串（字符串内不转义）

字符串前缀为‘R’或‘r’

```
print(r'hello\tworld') # 输出： hello\tworld
```

4. 变量及其赋值



(1) 简单赋值

```
a = 1
```

- 多目标赋值 `a=b=c=1` # 这种情况下 `a`、`b`、`c` 都是引用同一个变量
- 这样会报错 `a=(b=c=1)` `a=(b=1)`

(2) 序列赋值

- 左边是元组、列表表示的多个变量，右侧是元组、列表或字符串等序列表示的值。
- 序列元素个数必须和变量个数相等，否则会出错。
- 在变量名前使用“*”创建列表对象引用。

```
a, b = 1, 2
```

```
# 省略括号的元组赋值 (c, d) = (2, 3)
```

```
# 元组赋值 [e, f] = [4, '5']
```

```
# 列表赋值 (g, h) = ['6', 7]
```

```
# 元组和列表可以交叉赋值 (x, y, z) = 'abc' #
```

```
字符串赋值, x = 'a', y = 'b', z = 'c' (i, j) = (8, 9, 10)
```

```
# 这是错误的, 变量和值的个数必须相等
```

- 在变量名前使用“*”创建列表对象引用

```
x, *y = 'abcd' print(x, y)
```

```
# 输出: a ['b', 'c', 'd']
```

四、运算符和表达式

- 包括：算术运算符、关系运算符、字符串运算符、逻辑运算符。

1. 算术运算符和表达式

- 算术运算符包括：加(+)、减(-)、乘(*)、除(/)、取余(%)、整除(//)、幂运算(**)

`a+=1` 和 `a=a+1` 等价，`a-=1`、`a//=2` 等也类似



要注意这种复杂的表达式的运算优先级

```
int(8 * math. sqrt(36) * 10 ** (-2) *10 + 0. 5) /10
```

运算顺序如下：

```
int(8 * 6 * 10 ** (-2) * 10 + 0. 5) /10
```

```
10**(2) =0. 01
```

```
8 * 6 = 48
```

```
int(48 * 0. 01 * 10 + 0. 5 ) /10
```

```
int(5. 3) /10
```

```
5/10
```

```
0. 5
```

2. 逻辑运算符

- and （逻辑与） ， or （逻辑或） ， not （逻辑非）

3. 关系运算符

- == （等于） 、 != （不等于） 、 <> （不等于） 、 > （大于） 、 < （小于） 、 >= （大于等于） 、 <= （小于等于）

4. 运算符的优先级

- 最高优先级的三个运算符（优先级由高到低） ： ** 幂运算、 ~ 按位取反、 - 负号
- 最低优先级的几个运算符（优先级由高到低） ： | 按位或、< > <= >= == != 关系运算符、 not and or 逻辑运算符

5. 字符串运算符

（1）下面这些运算对于列表、元组都有效（因为它们同属序列类型）

字符串连接 (+)

```
print(' a' +' b' ) # 输出： ab
```

重复字符串 (*)

```
print(' a' *3) # 输出： aaa
```



索引字符串 ([])

```
a='hello' ; print(a[1]) ; # 输出: e
```

截取字符串 ([:])

```
print(a[1:4]) # 输出: ell
```

成员运算符 (in)

```
print('e' in a) # 输出: True
```

成员运算符 (not in)

```
print('e' not in a) # 输出: False
```

Raw 字符串 (r/R)

```
print(R'he\tllo') # 输出: hello\nllo
```

格式字符串 (%)

```
print('hello %s%s' %('wor', 'ld')) # 输出: hello world
```

(2) 格式化

%c (转换为字符)

```
print(' %c' %('hello')) # 报错, 必须是 ASCII 码值或者一个字符, 否则会出错
```

%r (使用 repr() 进行字符串转换)

```
print(' %r' %('hello')) # 输出: 'hello'
```

%s (使用 str() 进行字符串转换)

```
print(' %s' %('hello')) # 输出: hello
```

.format() 格式化

```
print('a={}' .format('a')) # 输出: a=a
```

repr() 函数和 str() 函数的区别就在于接受值和返回值不同

repr() 函数和 str() 函数, 分别会调用输入对象的__repr__()、__str__()

特殊方法

%d 或%i (转换为有符号十进制数)

```
print('%d' %(-10)) # 输出: -10
```



%u (转换为无符号十进制数)

print(' %u' %(-10)) # 输出: -10

有无符号是指在二进制下, 最高位用来表示实际的数或是单纯表示正负

%o (转换为无符号八进制数)

print(' %o' %(100)) # 输出: 144

%x 或%X (转换为无符号十六进制数)

print(' %x' %(100)) # 输出: 64

%e 或%E (转换成科学计数法)

print(' %e' %(1000)) # 输出: 1. 000000e+03

%f 或%F

print(' %f' %(100)) # 输出: 100. 000000

(3) 格式化操作辅助符

print(' 开始%10. 2f 结束' %(7. 2222)) # 输出: 开始 7. 22

%10. 2f 表示: 最小总长度为 10, 不足用前导空格补齐, 长度大于等于 10 则正常显示 (这里的长度不包括小数点)

6. 位运算符

• 异或: 二进制数逐位对比相同为 0, 不同为 1

$10^2=8$ 1010 异或 0010 结果为: 1000

运算符	说明
&	按位与
	按位或
^	按位异或
~	按位去反
<<	按位左移
>>	按位右移



五、python 常用函数

1. 数据类型转换函数

重点掌握加粗的函数

函数名	说明
int(str)	将字符串 str 转换为<u>整数</u>
long(str)	将字符串 str 转换为<u>长整型整数</u>
float(str)	将字符串 str 转换为<u>浮点数</u>
eval(str)	将字符串 str 转换为<u>有效表达式</u>并返回表达式计算后的对象
str(x)	将数值 x 转换为<u>字符串</u>
repr(obj)	将<u>对象</u>obj 转换为一个<u>字符串</u>
chr(x)	将整数 x 转换为对应的<u>ASCII 字符</u>
ord(x)	将字符 x 转换为对应的<u>ASCII 码</u>
hex(x)	将一个整数 x 转换为一个<u>十六进制字符串</u>
oct(x)	将一个整数 x 转换为一个<u>八进制字符串</u>
tuple(squ)	将一个序列 squ 转换为一个<u>元组</u>
list(squ)	将一个序列 squ 转换为<u>列表</u>
set(squ)	将一个序列 squ 转换为可变<u>集合</u>



<code>dict(squ)</code>	创建一个字典， <code>squ</code> 是一个序列 (key, value) 元组
<code>len(obj)</code>	返回对象的长度（字符个数、 列表元素个数、 字典 key 个数）

2. 数学函数

函数名	说明
<code>abs(x)</code>	返回数值 <code>x</code> 的绝对值
<code>exp(x)</code>	返回 <code>e</code> 的 <code>x</code> 次幂
<code>fabs(x)</code>	返回数字的绝对值
<code>log10(x)</code>	返回以 10 为底的 <code>x</code> 的对数
<code>pow(x, y)</code>	返回 <code>x</code> 的 <code>y</code> 次幂
<code>floor(x)</code>	<code>x</code> 向下取整（小于等于 <code>x</code> ）
<code>ceil(x)</code>	<code>x</code> 向上取整（大于等于 <code>x</code> ）
<code>fmod(x, y)</code>	求 <code>x/y</code> 的余数
<code>sin(x)</code> <code>cos(x)</code> . . .	返回 <code>x</code> 的三角函数值

六、python 数据结构

- python 常用三种数据结构：序列、映射、集合
- 列表和元组有什么相同点和不同点

1. 字符串

字符串是不可变的序列， 不可以通过 `str[n] = chr` 来改变字符串

(1) 字符串的切片（左闭右开）

```
word = 'hello world' print( word [0: 5])
```

```
# 输出： helloprint( word [: 5])
```



```
# 输出: helloprint( word [1: ])
# 输出: ello worldprint( word [: ])
# 输出: helloworldprint( word [0: 5: 2])
# 输出: hloprint( word [2: -2])
# 输出: lloworprint( word [-2: 2])
# 输出空串
```

(2) 字符串转列表

- 可以通过 `list()` 函数直接将字符串中的每个字符转换为一个列表元素，也可以通过 `split()` 方法根据指定的分割符分割元素（默认以空格分割）。

```
word = ' hello world' print(list( word ))
# 输出: [' h' , ' e' , ' l' , ' l' , ' o' , ' ' , ' w' , ' o' , ' r' ,
' l' , ' d' ]print( word . split ( ) )
# 输出: [' hello' , ' world' ]
```

(3) 列表转字符串

- 可以通过 `''.join()` 方法将字符串列表连接起来形成一个新的字符串

```
word = [' h' , ' e' , ' l' , ' l' , ' o' ]
words = ' ' . join ( word ) print( words )
# 输出: hello
```

(4) 字符串查找

- 可以通过 `in` 关键字查找也可以通过字符串的 `find()` 方法，`in` 关键字返回 `True` 或 `False`，`find()` 方法返回查找字符串的开始下标或-1，通过 `index()` 方法查找当没有查找到时会报错。

```
word = ' hello world' print(' llo' in word )
# 输出: Trueprint( word . find ( ' llo' ) )
# 输出: 2print( ' ok' in word )
# 输出: Falseprint( word . find ( ' 0k' ) )
# 输出: -1print( word . index ( ' ok' ) )
```



报错

#带开始和结束范围的查找 print(word . find (' llo' , 5, -1))

输出: -1print(word . index (' llo' , 0, 5))

输出: 2

(5) 字符串替换

• replace()方法替换指定的字符串

str1 = ' hello world' print(str1 . replace (' world' , ' yznu'))

输出: helloyznu

指定参数来限制替换的次数, 不指定参数默认替换所有匹配的字符串

可以看到下面的语句只替换了一个 1

print(str1 . replace (' l' , ' yes' , 1))

输出: heyeslo world

2. 列表

(1) 列表的表示方法

[1, 2, 3, 4]

(2) 创建列表

创建空列表、 列表生成式、 元组和字符串转列表

创建一个空列表

lst = []

直接将列表赋值给变量

lst = [' h' , ' e' , ' l' , ' l' , ' o']

元组转列表

lst = list((1, 2, 3, 4)) print(lst)

输出: [1, 2, 3, 4]

字符串转列表

lst = list(' hello') print(lst)

输出: [' h' , ' e' , ' l' , ' l' , ' o']



字符串分割成列表

```
lst = 'hello world' . split () print ( lst )
```

输出: ['hello' , ' world']

range() 生成列表

```
lst = list ( range (1, 6) ) print (lst)
```

输出: 【1, 2, 3, 4, 5】# 列表生成式

```
lst=【x**2 for x in range (1, 6) 】 print (ist)
```

#输出: 【1, 4, 9, 16, 25】# 复杂列表生成式

```
lst=【x for x in lst if x>1 and x<25】 print (ist)
```

#输出: 【4, 9, 16】

(3) 列表元素删除

- del(), pop(), remove()

del 语句删除

```
lst = 【1, 2, 3, 4, 5, 6】 del (ist 【5】) print (ist)
```

#输出: 【1, 2, 3, 4, 5】

#pop 方法删除并返回指定下标的元素 print(ist.pop(4))#输出: 5print(lst)

#输出: 【1, 2, 3, 4】

remove 方法删除列表中的首次出现的某个值 lst.remove (4) print (lst)

#输出: 【1, 2, 3】

(4) 列表分片

- 使用【:】对列表分片和对字符串分片相同, 列表还能分片赋值

列表分片赋值

```
lst = [1,2,3,4,5]
```

```
lst 【1: 4】 = 【0, 0, 0】
```

#切片个数和赋值个数一样, 否则报错 print (ist)

#输出: 【1, 0, 0, 0, 5】

(5) 字符串转列表和列表转字符串



- 在上面字符串那一节已经写过

(6) 列表常用的函数和方法

- `append()`, `count()`, `extend()`, `index()`, `sort()`, `len()`
- 注意区别 `extend()` 和 `append()` 的用法
- 注意区别 `sort()` 和 `sorted()` 的用法
- `count()` 是非递归的统计一个元素出现的次数，也就是不会深入第二层

函数和方法	说明
<code>.append()</code>	末尾追加一个元素
<code>.count()</code>	统计某元素出现的次数（非递归式）
<code>.extend()</code>	以另一个序列的元素值去扩充原列表
<code>.insert()</code>	在指定下标处插入一个元素
<code>.pop()</code>	删除指定下标的元素并返回该元素
<code>.reverse()</code>	翻转列表
<code>.sort()</code>	对列表进行排序
<code>.index()</code>	找到指定值第一次出现的下标（找不到报错）
<code>.remove()</code>	移除在列表中第一次出现的指定的值
<code>len()</code>	返回列表的元素个数
<code>max()</code>	返回最大值
<code>min()</code>	返回最小值

`lst = [1, 3, 4, 5]` # `append` 向列表中添加一个元素

`lst.append([6, 7])` `print( lst )`

输出: `[1, 3, 4, 5, [6, 7]]`

`extend` 用序列扩充列表

`lst.extend([8, 9, 10])` `print( lst )`

输出: `[1, 3, 4, 5, [6, 7], 8, 9, 10]`

`insert` 方法在指定下标处插入一个值



```
lst . insert (1, 2) print( lst )
# 输出: [1, 2, 3, 4, 5, [6, 7], 8, 9, 10]
# count 统计一个元素在列表中出现的次数
lst = [1, [2, 3], 2, 3]print( lst . count (2) )
# 输出: 1
```

3. 元组

- 元组的元素定义后不能修改

(1) 元组的表示方法

(1, 2, 3, 4)

(2) 元组的创建

- 创建空元组、 列表和字符串转元组
- 元组不能直接使用列表的方法排序（因为不可修改）

创建一个空元组

```
tup = ()
```

直接将元组赋值给变量

```
tup = (1, 2, 3)
```

列表转元组

```
tup = tuple([1, 2, 3])
```

输出: (1, 2, 3) print(tup)

字符串转元组

```
tup = tuple(' hello' ) print( tup )
```

输出: (' h' , ' e' , ' l' , ' l' , ' o')

(3) 元组和列表的相同点和不同点

相同点:

- o 元组和列表都属于序列
- o 元组和列表都支持相同操作符， 包括 + （连接） 、 * （重复）、 [] 索引、 [:]切片、 in 、 not in 等



不同点:

- o 元组是不可变的对象
- o 对列表进行的修改元素的操作, 在元组上不可以(例如 del 删除一个元组中的元素会出错, 只能将整个元组删掉, 不能只删某个元素)

4. 字典

- 字典是映射类型
- 字典的 key 值必须是可哈希的(也可以说是不可修改的)

(1) 字典的表示方法

```
{ key: value, key: value}
{ ' name' : ' allen' , ' age' : 18}
```

(2) 创建字典

- 创建空字典(使用空大括号默认创建字典, 集合使用 set())
- fromkeys() 创建字典、dict() 创建字典

创建一个空字典

```
dict1 = {}
```

直接将字典赋值给变量

```
dict1 = {' one' :1, ' two' :2, ' three' :3} print( dict1 )
```

输出: {' one' : 1, ' two' : 2, ' three' : 3}

dict 创建字典(对比下面几种创建字典的形式)

#方式 1

```
dict1 = dict( one =1, two =2, three =3)
```

#这样创建字典, key 不加引号

```
print( dict1 )
```

输出: {' one' : 1, ' two' : 2, ' three' : 3}

#方式 2

```
temp = [(' two' , 2) , (' one' , 1) , (' three' , 3) ]
```

```
dict1 = dict( temp ) print( dict1 )
```




```
# 输出: {' two' : 2, ' one' : 1, ' three' : 3}
```

#方式 3

```
temp = [[' two' , 2],[' one' , 1],[' three' , 3]]
```

```
dict1 = dict( temp ) print( dict1 )
```

```
# 输出: {' two' : 2, ' one' : 1, ' three' : 3}
```

#方式 4

```
temp = ([' two' , 2],[' one' , 1],[' three' , 3])
```

```
dict1 = dict( temp ) print( dict1 )
```

```
# 输出: {' two' : 2, ' one' : 1, ' three' : 3}
```

使用 fromkeys 创建字典

```
sub ject = [' Python' , ' 操作系统' , ' 计算机网络' ]
```

```
scores = dict. fromkeys ( subject ,99) print( scores )
```

```
# 输出: {' Python' : 99,' 操作系统' : 99, ' 计算机网络' : 99}
```

(3) 字典常用函数和方法

- dict(), values(), keys(), get(), clear(), pop(), in(), update(), copy()
- 注意区别 get() 和直接访问字典元素的区别

```
scores ={' Python' :99,' 操作系统' :99,' 计算机网络' :99}
```

```
#print(scores[' 大学英语' ])
```

```
# 这会报错, 因为不存在' 大学英语' 这个键
```

```
print( scores . get ( ' 大学英语' ) )
```

```
# 输出: None
```

```
# values 返回字典的所有值 print( scores . values () )
```

```
# 输出: dict_values([99,99, 99])
```

```
# keys 返回字典的所有键 print( scores . keys () )
```

```
# 输出: dict_keys([' Python' , ' 操作系统' , ' 计算机网络' ])
```

```
# items 返回字典的所有键和值 print( scores . items () )
```

```
# 输出: dict_items([(' Python' , 99) , (' 操作系统' , 99) , (' 计算机
```



```
网络' , 99) ]])
# 使用 keys 遍历 for k in scores . keys () :
print( k , end = ' ' ) ; # 输出: Python 操作系统 计算机网络
# 使用 values 遍历 for v in scores . values () :
print( v , end = ' ' ) ; # 输出: 99 99 99
# 使用 items 遍历 for k , v in scores . items () :
print( k , v , end = ' ; ' ) ; # 输出: Python 99; 操作系统 99; 计算机
网络 99;
# in 查询一个字典中是否包含指定键 print( ' Python' in scores )
# 输出: Trueprint( ' 数据结构' in scores )
# 输出: False
# update 更新字典, 如果包含相同的键将会覆盖原来的对应的那个键值
dict1 = { ' 语文' : 90, ' 数学' : 91 }
scores . update ( dict1 ) print( scores )
# 输出: { ' Python' : 99, ' 操作系统' : 99, ' 计算机网络' : 99, ' 语文' :
90, ' 数学' : 91 }
# 删除字典元素 del( scores [ ' 语文' ]) print( scores )
# 输出: { ' Python' : 99, ' 操作系统' : 99, ' 计算机网络' : 99, ' 数学' :
91 }
scores . pop ( ' 数学' ) print( scores )
# 输出: { ' Python' : 99, ' 操作系统' : 99, ' 计算机网络' : 99 }
# 清空字典
scores . clear () print( scores )
# 输出: {}
```

5. 集合

- 集合的元素不会有重复, 利用这个特性编程题中可以去重
- 集合内的元素只能是可哈希的数据类型 (也可以说是不可修改的), 无法存储





列表、字典、集合这些可变的数据类型

- 集合是无序的，所以每次输出时元素的排序顺序可能都不相同

(1) 创建集合

- 创建空集合不能直接用大括号，大括号默认创建空字典
- 将序列转为集合

创建一个空集合不能直接用大括号

```
set1 = set()
```

将集合直接赋值给变量

```
set1 = {' Python' , ' 计算机网络' , ' 操作系统' } print( set1 )
```

输出： {' 操作系统' , ' 计算机网络' , ' Python' }

列表转集合

```
set1 = set([' 操作系统' , ' 计算机网络' , ' Python' , ' 操作系统' ])  
print( set1 )
```

输出： {' Python' , ' 计算机网络' , ' 操作系统' }

字符串转集合

```
set1 = set(' 操作系统' ) print( set1 )
```

输出： {' 系' , ' 作' , ' 操' , ' 统' }

(2) 集合的一些操作

```
set1 = {' Python' , ' 计算机网络' , ' 操作系统' }
```

```
set2 = set([' 语文' , ' 操作系统' , ' 计算机网络' ]) # 集合中没有 + 的操作#
```

```
print(set1+set2) #这样会出错
```

差集 (set1 有但是 set 没有的元素) print(set1 - set2)

输出： {' Python' }

交集 (set1 和 set2 都有的元素) print(set1 & set2)

输出： {' 操作系统' , ' 计算机网络' }

并集 (set1 和 set2 所有的元素) print(set1 | set2)



学长学姐互助答疑群



期末突击课

```
# 输出: {' 语文' , ' 操作系统' , ' 计算机网络' , ' Python' }
```

```
# 不同时包含于 a 和 b 的元素 print( set1 ^ set2 )
```

(3) 集合的常用方法

```
set(), add(), update(), remove()
```

集合不支持用 del() 删除一个元素

```
set1 = set([' 操作系统' , ' 计算机网络' , ' Python' , ' 操作系统' ])
```

```
# add 向集合中添加一个元素
```

```
set1 . add ( ' 语文' ) print( set1 )
```

```
# 输出: {' 语文' , ' 计算机网络' , ' 操作系统' , ' Python' }
```

```
# remove 删除集合中一个元素
```

```
set1 . remove ( ' 语文' ) print( set1 )
```

```
# 输出: {' Python' , ' 操作系统' , ' 计算机网络' }
```

七、程序流程控制（记得语句后面的冒号：）

- 程序流程控制一定要注意语句后面的冒号和语句块之间的缩进

1. 分支选择

(1) 单分支

- if:

(2) 多分支

```
if:
```

```
.....
```

```
elif:
```

```
.....
```

```
else:
```

```
.....
```

(3) 程序例子



```
# 判断一个人的身材是否正常
height = float(input("输入身高（米）："))
weight = float(input("输入体重（千克）："))
bmi = weight / (height * height) #计算 BMI 指数
if bmi < 18.5:
    print("BMI 指数为：" + str(bmi))
    print("体重过轻")
elif bmi >= 18.5 and bmi < 24.9:
    print("BMI 指数为：" + str(bmi))
    print("正常范围，注意保持")
elif bmi >= 24.9 and bmi < 29.9:
    print("BMI 指数为：" + str(bmi))
    print("体重过重")
else:
    print("BMI 指数为：" + str(bmi))
    print("肥胖")
```

(4) 三目运算

- $\text{max} = a \text{ if } a > b \text{ else } b$ 为真 $\text{max}=a$, 否则 $\text{max}=b$

```
a = int(input('请输入 a: ')) # 假设输入 10
b = int(input('请输入 b: ')) # 假设输入 15
if a > b :max = a
else: max = b
print(max) # 输出: 15
max = a if a > b else b
print(max) # 输出: 15
```

2. 循环结构

(1) while 循环

- while 循环的格式

```
# 循环的初始化条件
num = 1 # 当 num 小于 100 时，会一直执行循环体
```



```
while num <=100:
    print("num=", num )
```

迭代语句

```
num +=1print("循环结束!")
```

(2) for 循环

- for 循环的格式

- for 循环遍历字典、集合、列表、字符串、元组

```
subjects = [' Python' , ' 操作系统' , ' 网络' ]# 循环遍历列表
```

```
lfor i in subjects :
```

```
print( i , end = ' ' )
```

```
# 输出: Python 操作系统 网络 print(' ' ) # 循环遍历列表
```

```
2for i in range(len( subjects ) ) :
```

```
print( subjects [ i ], end = ' ' )
```

```
# 输出: Python 操作系统 网络 print(' ' )
```

#遍历字符串，元组也类似，就不写了

循环遍历列表 1 这种形式也可以用于字典、集合，但循环遍历列表 2 这种形式不行

集合不能使用索引（因为它是无序的），所以使用 set[i]这种形式输出会报错

#字典必须用键去索引值， dict[key]这种形式

- 列表生成式

- range()函数与 for 循环的使用

range 函数的格式如下：

```
range (start, end, step)
```

start: 计数从 start 开始。默认是从 0 开始。例如 range (5) 等价于 range (0, 5)

end: 计数到 end 结束，但不包括 end 例如： range (0, 5) 是 0,



1, 2, 3, 4 没有 5

step: 步长, 默认为 1。例如: range(0, 5) 等价于 range(0, 5, 1)

列表生成式

```
lst = [ x for x in range(2, 11, 2) ]print( lst )
```

输出: [2, 4, 6, 8, 10]

```
lst = [ x *2 for x in range(2, 11, 2) ]print( lst )
```

输出: [4, 8, 12, 16, 20]

直接循环 range 函数

```
for i in range(2, 11, 2) : print( i , end = ' ' )
```

输出: 2 4 6 8 10 print(' ')

```
for i in range(11) : print( i , end = ' ' )
```

输出: 0 1 2 3 4 5 6 7 8 9 10 print(' ')

(3) break 和 continue 关键字

输出 1-50 之间的偶数# 循环的初始化条件

```
num = 1 while num <= 100 :
```

```
    if( num >50) :
```

当 num>50 跳出循环

```
    break
```

```
    if( num %2!=0) :
```

num 为奇数跳过输出, 直接加一进入下一次循环

```
        num += 1
```

```
        continue
```

```
print("num=", num )
```

迭代语句

```
num += 1print("循环结束!")
```

八、一些应用





1. 随机抽取

- random. sample()

- 从一个序列中随机抽出指定数量的元素，并以列表形式返回

```
import random
str1 = ' abcde'
lst = [1, 2, 3, 4, 5]
tuple1 = (1, 2, 3, 4, 5)
# 随机从字符串中抽取 5 个字符组成新的列表
rand_str = random . sample ( str1 ,5) print( rand_str )
# 随机从列表中抽取 5 个元素组成新的列表
rand_lst = random . sample ( lst , 5) print( rand_lst )
# 随机从元组中抽取 5 个元素组成新的列表
rand_tuple = random . sample ( tuple1 , 5) print( rand_tuple )
```

- random. choice()

- 从一个序列中随机抽取一个元素

```
import random
str1 = ' abcde'
lst = [1, 2, 3, 4, 5]
tuple1 = (1, 2, 3, 4, 5)
# 从字符串中随机抽取一个字符
a = random . choice ( str1 ) print( a )
# 从元组中随机抽取一个元素
a = random . choice ( tuple1 ) print( a )
```

2. 去重

- 利用集合去重

```
set1 = set()
```



学长学姐互助答疑群



期末突击课

```
set1 . add (1)
set1 . add (2)
set1 . add (3)
set1 . add (1)
set1 . add (2)
set1 . add (3)
print( set1 )
# 输出: {1, 2, 3}
```

- 利用循环遍历列表去重

```
list1 = [2, 1, 3, 6, 2, 1]
temp = []
for i in list1 :
    if not i in temp :
        temp . append ( i )
print( temp )
# 输出: [2, 1, 3, 6]
```

3. 一些特殊的字符串判断操作

```
str1 = ' hello'
str2 = ' 1234'
str3 = ' hello1234'
str4 = ' \n \t'
# 判断所有字符都是字母 print( str1 . isalpha () )
# 输出: True
# 判断所有字符都是数字 print( str2 . isdigit () )
# 输出: True
# 判断所有字符都是 数字或者字母 print( str3 . isalnum () )
# 输出: True
# 判断所有字符都是空白字符 (不只是空格) print( str4 . isspace () )
# 输出: True
```



4. 字符串转字节

```
str1 = ' yznu' print( str1 )
```

输出: yznu

下面的输出都是字节类型#

第一种

```
print(b' yznu' )
```

输出: b' yznu'

第二种

```
str2 = bytes( str1 , encoding = ' utf-8' ) print( str2 )
```

输出: b' yznu'

第三种 注意: encode 方法并不会将原来的字符串变为字节, 而是重新复制一份字节

```
str2 = str1 . encode ( ' utf-8' ) print( str2 )
```

输出: b' yznu'

5. 字节转字符串

```
byte = b' yznu'
```

这是字节

第一种

```
str1 = str( byte , encoding ="utf-8") print( str1 )
```

输出: yznu

第二种

```
str1 = bytes. decode ( byte ) print( str1 )
```

输出: yznu

